banana-brain: training of the raw brain network

Date: October 24, 2026 · **Sources**: `lab/src/model.js` (JavaScript copy of the C++ rules), `lab/src/lab-tasks.js`, `src/brain/network/RawBrainNetwork.cpp`, benchmark scripts of this session

> **How to read this document**
>
> 1. The symptoms: plateaus, zigzag, jumps of the error, frozen networks, worse results with many neurons.
> 2. Their causes, measured on many networks (not on one seed).
> 3. The options added to the lab and to `RawBrainNetwork`, and their effect before / after.
> 4. The links and steps caps, and the new defaults chosen from the measures.
>
> Every number comes from the JavaScript simulation of the lab, which follows the rules of the C++ code (same init, forward, backpropagation through time and link changes). A "solved" network has every sample on the right side of 0.5. Unless written otherwise: task XOR, 13 hidden neurons (16 in all), width 8, height 5, links cap 6, 8 steps, sigmoid, `rand(-3, 3)`, `weightMin` 0.01, `weightMax` 5, left to right links, 600 rounds, 100 networks of random seeds. **Not included**: the GPU kernels (the new options run on the host), the layered network.

Contents

1. Summary
2. Symptoms and causes
3. Options and their effect
4. Large networks
5. Your two frozen networks
6. Links and steps caps
7. What changed in the code
8. Recommended settings
9. Sources

1. Summary

Point	Conclusion
XOR solved	**16 %** of the networks before, **50 %** with momentum 0.9, rate 1, the path at the init, the filter on every input and the low new links
Plateau at 0.25 - 0.33	the XOR local minimum: 3 or 2 samples of 4 right; training longer changes nothing (600 or 3000 rounds: same rate of success)
Zigzag	one update per sample with a high rate: each sample pulls the weights its way, the next one back
Jumps of the error	mostly a weight moving by more than 1 in one round (high rate), sometimes a new link with the size of its neighbours
Frozen network (flat line after a jump)	the weights into the output grow far beyond `weightMax` (no free slot to link, nothing caps them): the output saturates at 0 for every sample, its slope is ~1e-9 and nothing can move again
Many neurons	the filter only checked the nearest input: with 60 - 240 hidden neurons, 1 network of 4 has both inputs reaching the output; AND falls from 70 % to 10 % solved
Frozen output	`directOutput` (the score on the sum of the output without the sigmoid slope) keeps a saturated output alive: your two frozen networks end at **0.016** and **0.075** instead of 0.259, but on 30 CIRCLE networks it is not better on average (3 % solved, many jumps): a rescue, not a general fix
CIRCLE	hard for 13 neurons with a cap of 6: at best **17 %** solved (mini-batch + momentum 0.9), 0 % before
Caps	the define at 1.0 links every neuron to every other one: best on 16 neurons (XOR 80 %), but on 63 neurons 80 - 85 % of the networks freeze (a sum of 62 inputs saturates) and 120 hidden neurons were too slow to measure; 0.1 gives 1 link under 20 neurons (nothing learns); `2 log2 n + 2` for both caps: 70 % / 25 % (AND / XOR) on 60 hidden

2. Symptoms and causes

2.1 Plateau and zigzag

1. **The XOR local minimum.** On 60 networks of your configuration, 56 stop before solving: 33 with 3 samples of 4 right (mean error about 0.27), 23 with 2 of 4. The network computes OR, AND or one input.
2. **Not a matter of time.** 30 networks where both inputs reach the output: 6 solved in 600 rounds, still 6 in 3000 rounds.
3. **The zigzag** is the online update: one change per sample, rate 10, score up to 10. It disappears with a smaller rate, the momentum (smoothed steps) or the mini-batch (one change per round).

2.2 Jumps of the error

Over 60 networks x 600 rounds, 11 jumps of more than +0.05 in one round: 7 come with a weight moving by more than 1 in the round, 5 with a link removed or added. No jump cut an input from the output.

2.3 Frozen networks

The flat line after a jump (your two screenshots) is a **saturated output**. In your first case, at round 274 the output is 0.000 for the 81 samples, its inward weights are -14 and -16 and its bias +10, far beyond `weightMax` 5. `weightMax` only adds a link when a slot is free: with a full cap the weight keeps growing. The slope of the sigmoid is then ~1e-9: the gradient is 0 for ever, the error stays at the share of the samples at 1 (0.259 on CIRCLE).

> **Key takeaway**: capping each weight is not enough (6 links at 5 plus a bias of 5 still saturate the sigmoid); the score must reach the output without its slope (`directOutput`).

CIRCLE, 10 hidden, 50 networks	Final error	Jumps / network	Frozen	Links over `weightMax`
before (rate 10)	0.274	3.4	8 %	3.4
rate 3 (your screenshot)	0.263	2.9	8 %	4.5
rate 3 + clamp	0.329	0.0	8 %	0.0
after	0.255	14.1	28 %	4.1
after + clamp	0.276	15.3	22 %	0.0
after + clamp, weightMax 10	0.274	4.5	34 %	0.0

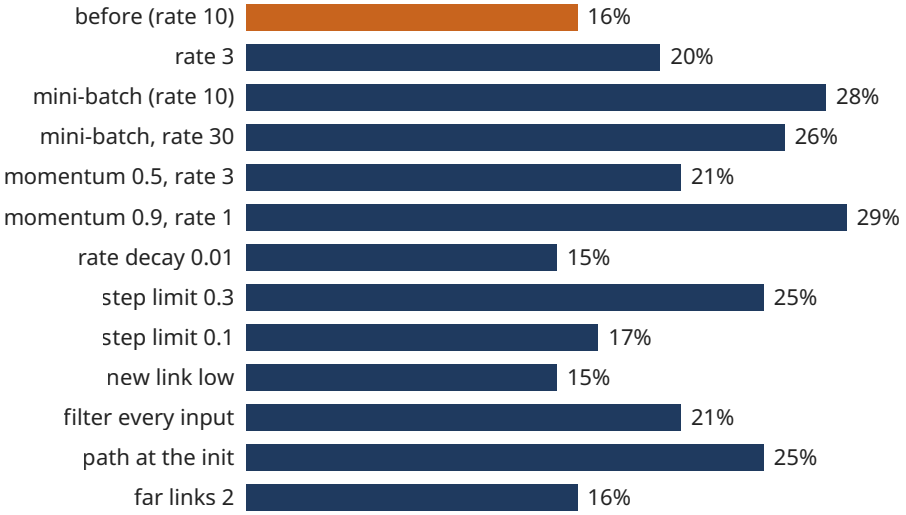
2.4 Inputs losing their path

Before, 24 % of the networks end with an input that has no path to the output any more. The protection "never cut the last path" only checked the nearest input: it now checks every pair input -> output (lab), and the filter of Regenerate / batch needs every input.

3. Options and their effect

3.1 Each option alone (XOR, 100 networks)

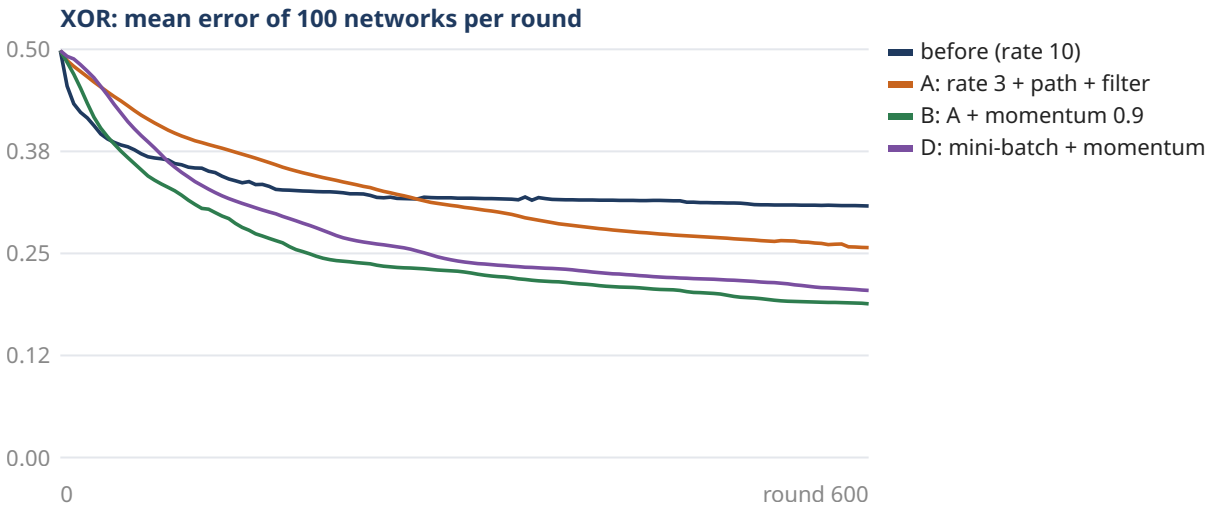
Networks solved, each option alone



Option	Solved	Final error	Jumps / network	Every input reaches the output	Median round solved
before (rate 10)	16 %	0.308	0.39	76 %	104
rate 3	20 %	0.310	0.16	75 %	166
mini-batch (rate 10)	28 %	0.329	0.15	75 %	104
mini-batch, rate 30	26 %	0.311	1.43	75 %	59
momentum 0.5, rate 3	21 %	0.300	0.12	75 %	74
momentum 0.9, rate 1	29 %	0.294	0.16	75 %	31
rate decay 0.01	15 %	0.325	0.11	75 %	96
step limit 0.3	25 %	0.300	0.10	77 %	85
step limit 0.1	17 %	0.368	0.09	76 %	171
new link low	15 %	0.306	0.14	76 %	80
filter every input	21 %	0.237	0.52	100 %	80
path at the init	25 %	0.272	0.78	100 %	128
far links 2	16 %	0.299	0.52	86 %	115

3.2 Combinations

Combination	Solved	Final error	Jumps / network	Median round solved
before (rate 10)	16 %	0.308	0.39	104
A: rate 3, limit 0.3, low, every input, path	32 %	0.257	0.19	199
B: A + momentum 0.9, rate 1	49 %	0.188	0.07	83
C: A + mini-batch, rate 10	40 %	0.250	0.07	137
D: mini-batch + momentum 0.9, rate 3	47 %	0.205	0.00	89
E: B without the step limit	50 %	0.186	0.02	95



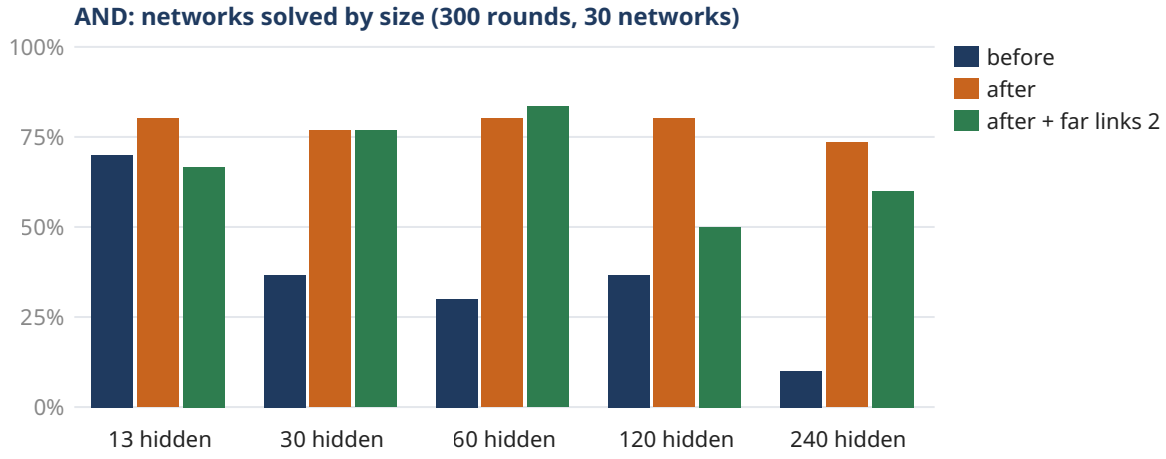
3.3 Score straight on the output (`directOutput`)

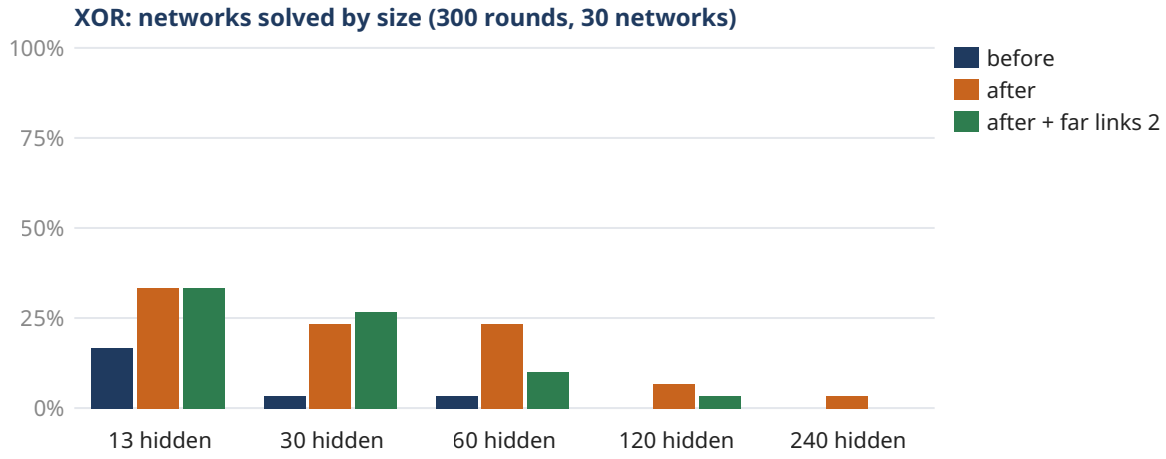
XOR, 100 networks	Solved	Final error	Jumps	Frozen	Median round solved
before (rate 10)	16 %	0.308	0.39	14 %	104
rate 3	31 %	0.262	0.00	8 %	213
rate 3, direct	42 %	0.238	0.27	10 %	38
rate 1, direct	43 %	0.240	0.01	9 %	81
rate 1, direct, limit 0.3	43 %	0.227	0.01	5 %	105
batch rate 10, momentum 0.9	47 %	0.185	0.20	13 %	51
batch rate 10, momentum 0.9, direct	14 %	0.383	4.67	32 %	49
batch rate 3, momentum 0.9, direct	43 %	0.221	0.22	21 %	39
momentum 0.9 rate 1 (XOR best)	49 %	0.188	0.07	3 %	83
momentum 0.9 rate 1, direct	51 %	0.194	0.05	4 %	36

`directOutput` removes the factor of the sigmoid slope (at most 0.25): the step is up to 4 times bigger, the rate must go down (1 to 3 online). With the mini-batch at rate 10 and the momentum together it overshoots on XOR, and on CIRCLE it makes many jumps.

CIRCLE, 30 networks	Solved	Final error	Jumps	Frozen
before (rate 10)	0 %	0.277	3.00	3 %
rate 3	7 %	0.206	7.20	3 %
rate 3, direct	0 %	0.280	5.17	20 %
rate 1, direct	3 %	0.245	5.50	3 %
rate 1, direct, limit 0.3	0 %	0.230	1.00	13 %
batch rate 10, momentum 0.9	17 %	0.227	1.63	43 %
batch rate 10, momentum 0.9, direct	0 %	0.347	67.83	43 %
batch rate 3, momentum 0.9, direct	17 %	0.232	4.13	23 %
momentum 0.9 rate 1 (XOR best)	0 %	0.260	12.83	27 %
momentum 0.9 rate 1, direct	0 %	0.259	4.00	77 %
batch rate 10	3 %	0.350	4.27	7 %
batch rate 10, direct	3 %	0.278	160.57	7 %
batch rate 30, direct	0 %	0.328	206.97	67 %

4. Large networks

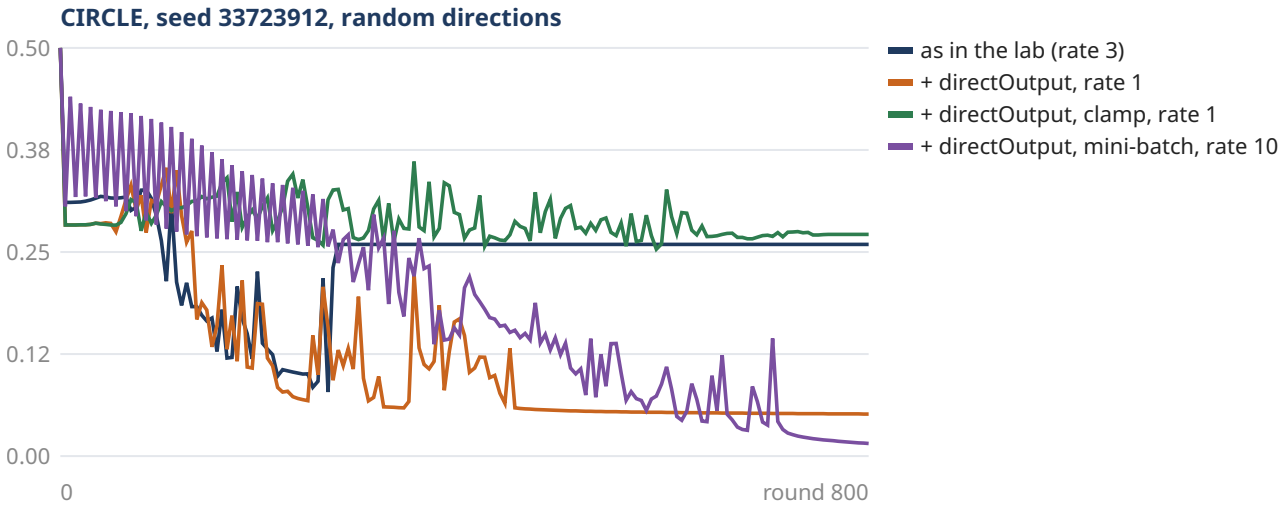


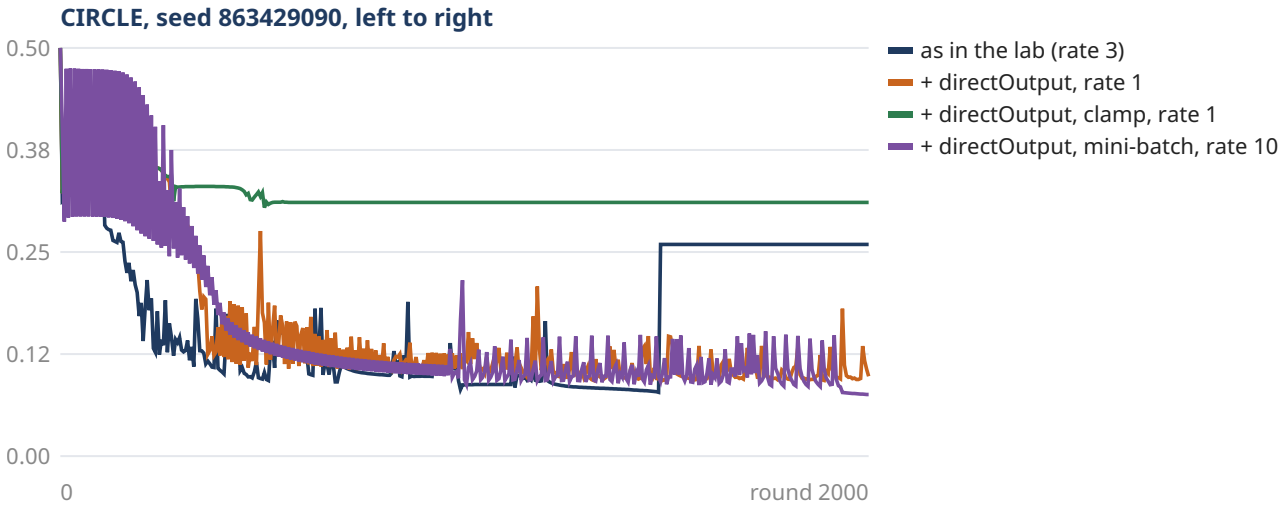


Hidden	Task	Solved before	Solved after	Every input before	Time / round
13	AND	70 %	80 %	83 %	0.36 ms
13	XOR	17 %	33 %	83 %	0.32 ms
30	AND	37 %	77 %	50 %	0.59 ms
30	XOR	3 %	23 %	43 %	0.60 ms
60	AND	30 %	80 %	43 %	1.16 ms
60	XOR	3 %	23 %	40 %	1.23 ms
120	AND	37 %	80 %	37 %	2.60 ms
120	XOR	0 %	7 %	30 %	3.76 ms
240	AND	10 %	73 %	23 %	6.43 ms
240	XOR	0 %	3 %	17 %	6.32 ms

"After" = momentum 0.9, rate 1, step limit 0.3, path at the init, filter on every input, low new links. The cause of the fall was the path of the second input; the far links (2 of the 6 slots to random neurons) do not help. The time per round grows with neurons x cap x steps.

5. Your two frozen networks





Variant	Case 1 final	Case 2 final
as in the lab (rate 3)	0.259	0.259
+ directOutput, rate 1	0.052	0.098
+ directOutput, clamp, rate 1	0.271	0.311
+ directOutput, mini-batch, rate 10	0.016	0.075

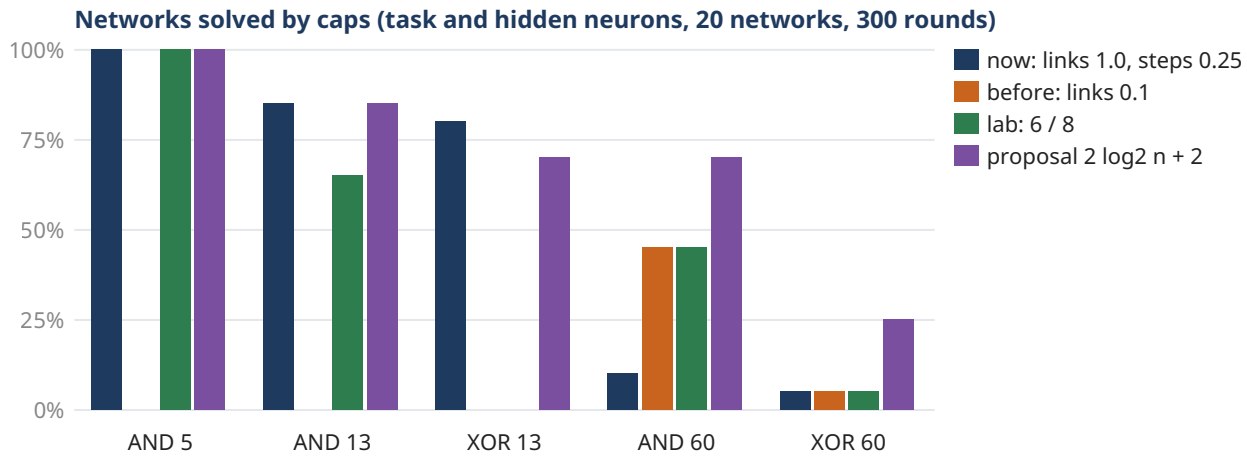
> **Key takeaway**: the clamp at `weightMax` 5 avoids the death but leaves too little room to draw the disc; `directOutput` keeps the output alive on these two seeds; on 30 CIRCLE networks the same settings are not better on average (part 3.3), so it is a rescue for a frozen output, not a default.

6. Links and steps caps

Chosen and applied: links `min(100 % n, 3 sqrt(n))`, steps `min(100 % n, 4 log2(n))` (part 6.3). The first tables compare the earlier candidates.

n	links hook (default)	cap at 1.0 (now)	cap at 0.1 (before)	steps hook	steps at 0.25	proposal links / steps (2 log2 n + 2)
5	78	4	1	9	1	4 / 6
8	130	7	1	12	2	7 / 8

n	links hook (default)	cap at 1.0 (now)	cap at 0.1 (before)	steps hook	steps at 0.25	proposal links / steps (2 log2 n + 2)
16	232	15	1	16	4	10 / 10
30	350	29	3	19	7	11 / 11
60	507	59	6	23	15	13 / 13
120	693	119	12	27	27	15 / 15
240	909	239	24	31	31	17 / 17
1000	1444	999	100	39	39	21 / 21
10000	2568	2568	1000	53	53	28 / 28

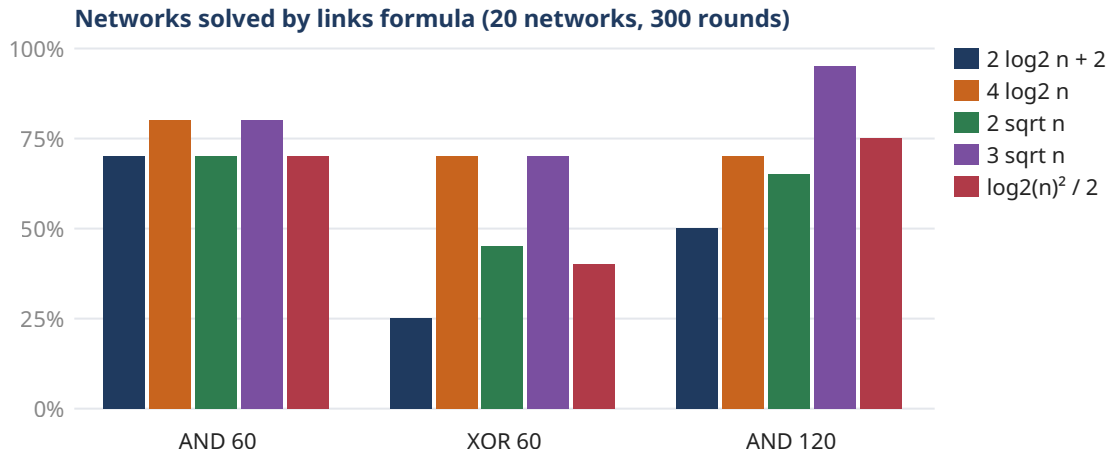


Hidden	Task	Caps	Solved	Frozen	Time / round
5	AND	current (links 1.0, steps 0.25) (cap 7, steps 2)	100 %	0 %	0.15 ms
5	AND	old links 0.1, steps 0.25 (cap 1, steps 2)	0 %	100 %	0.07 ms
5	AND	lab (links 6, steps 8) (cap 6, steps 8)	100 %	0 %	0.28 ms
5	AND	proposal (2 log2 n + 2 both) (cap 7, steps 8)	100 %	0 %	0.40 ms
13	AND	current (links 1.0, steps 0.25) (cap 15, steps 4)	85 %	0 %	0.85 ms
13	AND	old links 0.1, steps 0.25 (cap 1, steps 4)	0 %	100 %	0.16 ms
13	AND	lab (links 6, steps 8) (cap 6, steps 8)	65 %	10 %	0.89 ms
13	AND	proposal (2 log2 n + 2 both) (cap 10, steps 10)	85 %	10 %	1.62 ms
13	XOR	current (links 1.0, steps 0.25) (cap 15, steps 4)	80 %	0 %	0.89 ms
13	XOR	old links 0.1, steps 0.25 (cap 1, steps 4)	0 %	100 %	0.16 ms
13	XOR	lab (links 6, steps 8) (cap 6, steps 8)	0 %	20 %	0.63 ms
13	XOR	proposal (2 log2 n + 2 both) (cap 10, steps 10)	70 %	5 %	0.89 ms
60	AND	current (links 1.0, steps 0.25) (cap 62, steps 15)	10 %	80 %	52.46 ms
60	AND	old links 0.1, steps 0.25 (cap 6, steps 15)	45 %	10 %	3.36 ms
60	AND	lab (links 6, steps 8) (cap 6, steps 8)	45 %	10 %	1.82 ms
60	AND	proposal (2 log2 n + 2 both) (cap 13, steps 13)	70 %	0 %	5.17 ms
60	XOR	current (links 1.0, steps 0.25) (cap 62, steps 15)	5 %	85 %	39.03 ms
60	XOR	old links 0.1, steps 0.25 (cap 6, steps 15)	5 %	45 %	3.55 ms
60	XOR	lab (links 6, steps 8) (cap 6, steps 8)	5 %	45 %	2.17 ms
60	XOR	proposal (2 log2 n + 2 both) (cap 13, steps 13)	25 %	15 %	6.95 ms
120	AND	now (cap 122, steps 27)	n/a: too slow (about 20 times the proposal per round)	—	—

Training of this table: rate 3, path at the init, filter on every input, low new links. Steps beyond the depth of a left to right network change nothing (the "before" and "lab" rows are the same at 60 hidden).

6.2 Formulas that grow faster

n	2 log2 n + 2	4 log2 n	2 sqrt n	3 sqrt n	log2(n)² / 2
8	7	7	5	**7**	4
16	10	15	8	**12**	8
30	11	19	10	**16**	12
60	13	23	15	**23**	17
120	15	27	21	**32**	23
240	17	31	30	**46**	31
1000	21	39	63	**94**	49
10000	28	53	200	**300**	88
100000	35	66	632	**948**	137



Formula	Cap (60 / 120 hidden)	AND 60	XOR 60	AND 120	Time / round at 120
2 log2 n + 2	13 / 15	70 %	25 %	50 %	19.2 ms
4 log2 n	23 / 27	80 %	70 %	70 %	41.1 ms
2 sqrt n	15 / 22	70 %	45 %	65 %	27.7 ms
3 sqrt n	23 / 33	80 %	70 %	95 %	46.8 ms
log2(n)^2 / 2	17 / 24	70 %	40 %	75 %	30.2 ms

Steps `2 log2 n + 2` in this table. An init of the weights scaled by `sqrt(6 / cap)` helped AND (3 sqrt n at 120 hidden: 95 % -> 100 %) but hurt XOR (70 % -> 55 % at 60 hidden): not kept.

6.3 New defaults

n	Links (3 sqrt n, at most n - 1)	Steps (4 log2 n, at most n)
5	4	5
10	9	10
16	12	16
30	16	19
60	23	23
120	32	27

n	Links (3 sqrt n, at most n - 1)	Steps (4 log2 n, at most n)
240	46	31
1000	94	39
10000	300	53

> **Key takeaway**: `3 sqrt n` grows the fastest of the candidates and gave the best results on the large networks (AND 95 % at 120 hidden, XOR 70 % at 60); the steps keep `4 log2 n` without the 25 % limit.

7. What changed in the code

Where	Change
`RawBrainNetwork`	`leftToRight()` : links only from left to right, the left neighbours of each neuron computed once (init / load)
`RawBrainNetwork`	`momentum()` , `stepLimit()` , `clampWeights()` , `directOutput()` , `batch()` / `applyBatch()` / `pending()` , `lowNewLinks()` , `keepPaths()` : off by default, saved in the `.rbn` , run on the host
`RawBrainNetwork`	`farLinks()` and `pathInit()` at the init
tests	13 new tests (directions, file checks, momentum, step limit and clamp, mini-batch, new links, paths, kept paths, saved options, direct output, gpu = host)
lab	rate 3 by default, the options above, the protection of every input path, the filter on every input, the rate decay
`NetworkDefine.hpp`	links `BRAIN_LINKS_CAP_PERCENTAGE` 1.0, `BRAIN_LINKS_CAP_COMPUTING(n)` 3 sqrt(n); steps `BRAIN_STEPS_CAP_PERCENTAGE` 1.0 (hook 4 log2 n unchanged); the tests follow them, 146 pass

8. Recommended settings

Kind of task	Settings
Small logic (XOR, AND...)	momentum 0.9, rate 1, `pathInit(3)` , filter on every input, `lowNewLinks`
Many samples (curves, 2D)	mini-batch at rate 10 with momentum 0.9, `pathInit(3)` , more links than 6
Output frozen at a constant	`directOutput` with a rate of 1 - 3 (rescue)
Any	never a rate of 10 online; check that every input reaches the outputs (`keepPaths`); the new default caps instead of a cap of n - 1 above ~30 neurons

9. Sources

Code:

- `lab/src/model.js` , `lab/src/lab-tasks.js` : simulation and tasks
- `src/brain/network/RawBrainNetwork.cpp` : init, backpropagation, link changes

Benchmarks (job folder, not kept in the repository): `bench.js` groups `options` , `combo` , `size` , `freeze` , `direct` , `caps` ; `curves.js` ; `repro.js` .

Assumption: the JavaScript simulation behaves like the C++ code; the C++ options were checked by unit tests, not by the same benchmarks.